TERRAFORM FINAL TASK

AWS VPC and Ubuntu EC2 Using Terraform

Terraform Practical Documentation

1. Create VPC
2. Create Internet gateway
3. Create Custom Route Table
4. Create Subnet
5. Associate subnet with Route Table
6. Create Security Group to allow port 22.80,443
7. Create a network interface with an ip in the subnet that was created in step 4
8. Assign an elastic IP to the network interface created in step 7
9. Create Ubuntu server and install/enable apache2

1. Objective

To provision AWS networking resources and an Ubuntu EC2 instance using Terraform. Configure internet connectivity, assign an Elastic IP, allow SSH, HTTP, and HTTPS traffic, and automatically install and enable Apache2 using a Terraform configuration.

2. Prerequisites

1. AWS account.
2. Terraform installed on Windows.
3. AWS CLI installed.
4. AWS credentials configured.
5. VS Code installed.
6. An existing AWS EC2 key pair for SSH access.

Configure AWS credentials in PowerShell:

```
None
aws configure
```

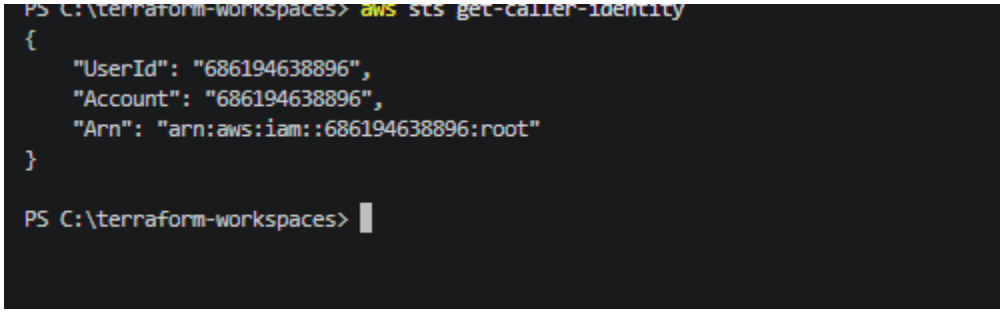
Enter:

- AWS Access Key ID
- AWS Secret Access Key

- Default region: `us-east-1`
- Default output format: `json`

Verify AWS access:

```
None
aws sts get-caller-identity
```



3. Create Project Folder

We will create one project folder containing all Terraform files.

Step 1: Open Windows PowerShell.

Execute:

```
None
mkdir terraform-vpc-project
```

Step 2: Open the project folder.

```
None
cd terraform-vpc-project
```

Step 3: Open this folder in VS Code.

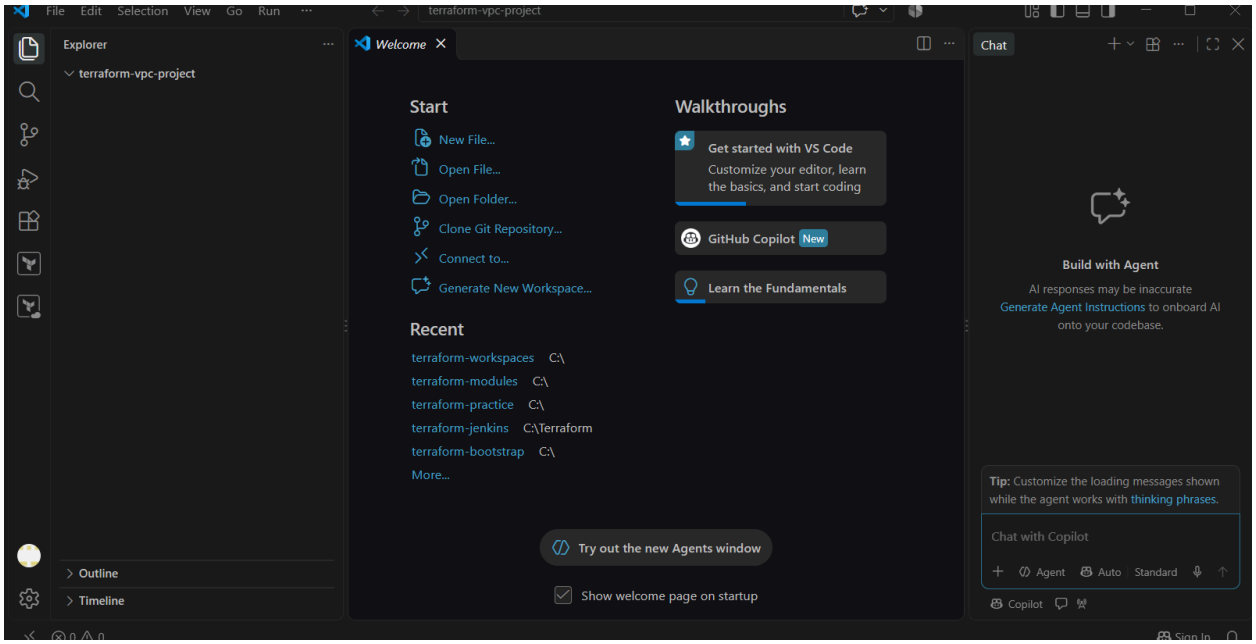
```
None
code .
```

```
PS C:\terraform-workspaces> mkdir terraform-vpc-project


    Directory: C:\terraform-workspaces


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        30-10-2026     12:01                terraform-vpc-project


PS C:\terraform-workspaces> cd terraform-vpc-project
PS C:\terraform-workspaces\terraform-vpc-project> cd ..
```



Step 4: Create the following files in VS Code Explorer.

Create these four files:

terraform-vpc-project

main.tf

Resources

variables.tf

Inputs

terraform.tfvars

Values

outputs.tf

Outputs

4. Create variables.tf

Purpose: This file contains the input variables used by Terraform.

In VS Code:

1. Right-click the project folder.
2. Select New File.
3. Enter `variables.tf`.
4. Paste the following code.

```
None
variable "region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}

variable "key_name" {
  description = "Existing AWS EC2 key pair name"
  type        = string
}

variable "my_ip" {
  description = "Your public IP address with CIDR"
  type        = string
}
```

Explanation:

- `region`: AWS region where resources are created.
- `instance_type`: EC2 instance size.
- `key_name`: Existing AWS key pair name used for SSH.
- `my_ip`: Your public IP address allowed to connect through SSH.

5. Create terraform.tfvars

Purpose: This file provides actual values for the variables.

1. Right-click the project folder.
2. Select New File.
3. Enter `terraform.tfvars`.
4. Paste the following:

```
None
region        = "us-east-1"
instance_type = "t2.micro"
key_name      = "YOUR-EXISTING-KEY-PAIR-NAME"
my_ip         = "YOUR-PUBLIC-IP/32"
```

Important:

- Replace `YOUR-EXISTING-KEY-PAIR-NAME` with your existing AWS EC2 key pair name.
- Replace `YOUR-PUBLIC-IP/32` with your actual public IP address, for example `203.0.113.10/32`. This example IP is only illustrative.

You can find your existing key pair name in AWS Console → EC2 → Network & Security → Key Pairs.

6. Create main.tf

Purpose: This is the main Terraform file. It contains all the AWS resources required for this practical.

1. Right-click the project folder.
2. Select New File.
3. Enter `main.tf`.
4. Paste the complete code below.

None

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# AWS Provider

provider "aws" {
  region = var.region
}

# 1. Create VPC

resource "aws_vpc" "my_vpc" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "My-VPC"
  }
}

# 2. Create Internet Gateway

resource "aws_internet_gateway" "my_igw" {
  vpc_id = aws_vpc.my_vpc.id

  tags = {
    Name = "My-IGW"
  }
}
```

```
# 3. Create Custom Route Table

resource "aws_route_table" "my_route_table" {
  vpc_id = aws_vpc.my_vpc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.my_igw.id
  }

  tags = {
    Name = "My-Route-Table"
  }
}

# 4. Create Subnet

resource "aws_subnet" "my_subnet" {
  vpc_id                  = aws_vpc.my_vpc.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "${var.region}a"
  map_public_ip_on_launch = false

  tags = {
    Name = "My-Subnet"
  }
}

# 5. Associate Subnet with Route Table

resource "aws_route_table_association" "my_association" {
  subnet_id      = aws_subnet.my_subnet.id
  route_table_id = aws_route_table.my_route_table.id
}
```

```
# 6. Create Security Group

resource "aws_security_group" "my_sg" {
  name        = "My-SG"
  description = "Allow SSH HTTP and HTTPS"
  vpc_id      = aws_vpc.my_vpc.id

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }

  tags = {
    Name = "My-SG"
  }
}

# 7. Create Network Interface

resource "aws_network_interface" "my_eni" {
  subnet_id       = aws_subnet.my_subnet.id
  private_ips     = ["10.0.1.10"]
  security_groups = [aws_security_group.my_sg.id]

  tags = {
    Name = "My-ENI"
  }
}

# 8. Allocate and Associate Elastic IP

resource "aws_eip" "my_eip" {
  domain                    = "vpc"
  network_interface         = aws_network_interface.my_eni.id
  associate_with_private_ip = "10.0.1.10"

  tags = {
    Name = "My-EIP"
  }
}

# Find Ubuntu 24.04 AMI

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
```

```
  filter {
    name   = "name"
    values =
["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }
}

# 9. Create Ubuntu EC2 Instance

resource "aws_instance" "ubuntu_server" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = var.instance_type
  key_name      = var.key_name

  network_interface {
    network_interface_id = aws_network_interface.my_eni.id
    device_index         = 0
  }

  user_data = <<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y apache2
    systemctl start apache2
    systemctl enable apache2
```

```
  EOF

  tags = {
    Name = "Ubuntu-Server"
  }
}
```

Understanding the main.tf code

Resource	Purpose
`aws_vpc`	Creates custom VPC
`aws_internet_gateway`	Provides internet connectivity
`aws_route_table`	Creates routing rules
`aws_subnet`	Creates subnet
`aws_route_table_association`	Associates subnet with route table
`aws_security_group`	Allows ports 22, 80, 443
`aws_network_interface`	Creates ENI with private IP
`aws_eip`	Allocates and associates Elastic IP

`data.aws_ami`	Finds the latest matching Ubuntu AMI
`aws_instance`	Creates Ubuntu EC2 and installs Apache2

Important: Terraform automatically understands resource dependencies through references such as `aws_vpc.my_vpc.id`. It creates the required resources in the correct dependency order.

7. Create outputs.tf

Purpose: This file displays the important information after Terraform creates the resources.

1. Create a new file named `outputs.tf`.
2. Paste the following code:

```
None
output "vpc_id" {
  value = aws_vpc.my_vpc.id
}

output "subnet_id" {
  value = aws_subnet.my_subnet.id
}

output "security_group_id" {
  value = aws_security_group.my_sg.id
}

output "network_interface_id" {
  value = aws_network_interface.my_eni.id
}

output "private_ip" {
  value = one(aws_network_interface.my_eni.private_ips)
}
```

```
output "elastic_ip" {
  value = aws_eip.my_eip.public_ip
}

output "instance_id" {
  value = aws_instance.ubuntu_server.id
}

output "apache_url" {
  value = "http://${aws_eip.my_eip.public_ip}"
}
```

Purpose of outputs:

- `vpc_id`: Displays the created VPC ID.
- `subnet_id`: Displays the subnet ID.
- `security_group_id`: Displays the security group ID.
- `network_interface_id`: Displays the ENI ID.
- `private_ip`: Displays the private IP address.
- `elastic_ip`: Displays the public Elastic IP.
- `instance_id`: Displays the EC2 instance ID.
- `apache_url`: Displays the Apache website URL.

8. Final Project Structure

Your VS Code Explorer should look like this:

terraform-vpc-project

main.tf

variables.tf

terraform.tfvars

outputs.tf

Before running Terraform, add this dependency inside the `aws_eip` resource in `main.tf`, after the `associate_with_private_ip` line:

```
None
depends_on = [aws_instance.ubuntu_server]
```

This ensures the EC2 instance has attached the network interface before Terraform associates the Elastic IP.

9. Execute Terraform Commands

Now we will execute Terraform commands using the VS Code terminal.

Where to go:

1. Open VS Code.
2. Click Terminal in the top menu.
3. Select New Terminal.
4. Make sure the terminal is inside `terraform-vpc-project`.

Step 1: Initialize Terraform

Run:

```
None
terraform init
```

Purpose: Downloads the AWS provider plugin and initializes the Terraform project.

Expected output:

```
None
Terraform has been successfully initialized!
```

```
PS C:\terraform-workspaces\terraform-vpc-project> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Step 2: Format Terraform Code

Run:

```
None
terraform fmt
```

Purpose: Automatically formats Terraform files into standard formatting.

```
PS C:\terraform-workspaces\terraform-vpc-project> terraform fmt
main.tf
```

Step 3: Validate Terraform Code

Run:

```
None
terraform validate
```

Purpose: Checks whether the Terraform configuration is syntactically correct and valid.

Expected output:

```
None
Success! The configuration is valid.
```

```
PS C:\terraform-workspaces\terraform-vpc-project> terraform validate
Success! The configuration is valid.
```

Step 4: Create Execution Plan

Run:

```
None
terraform plan
```

Purpose: Shows which AWS resources Terraform is going to create before making actual changes.

Expected output will show approximately:

```
None
Plan: 9 to add, 0 to change, 0 to destroy.
```

The exact count may vary because the Ubuntu AMI is a data source, not a resource.

```
Plan: 9 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + apache_url           = (known after apply)
  + elastic_ip           = (known after apply)
  + instance_id          = (known after apply)
  + network_interface_id = (known after apply)
  + private_ip           = "10.0.1.10"
  + security_group_id    = (known after apply)
  + subnet_id            = (known after apply)
  + vpc_id               = (known after apply)
```

Step 5: Create AWS Resources

Run:

```
None
terraform apply
```

Terraform displays the execution plan and asks for confirmation.

Type:

```
None
yes
```

Press Enter.

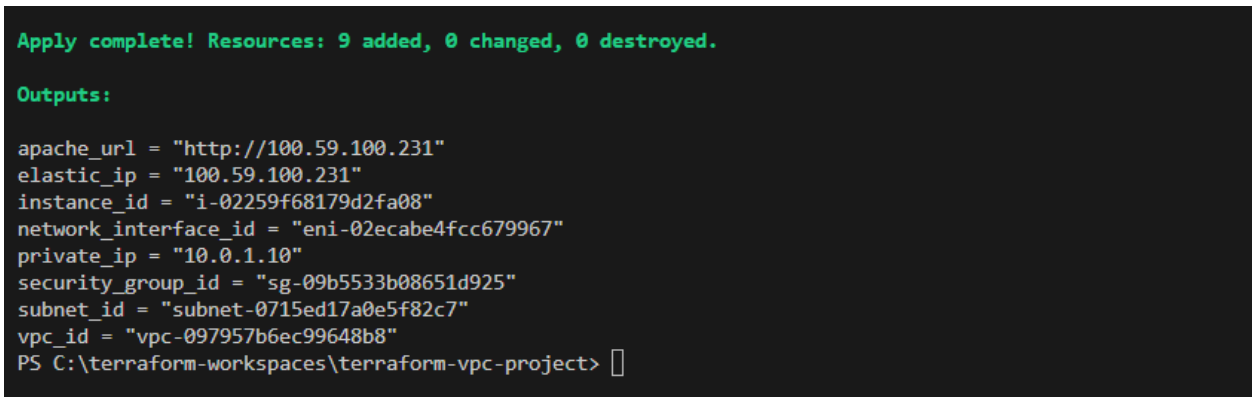
Purpose: Creates all AWS resources defined in `main.tf`.

Wait until Terraform completes the creation process.

Expected output:

```
None
Apply complete! Resources: 9 added, 0 changed, 0 destroyed.
```

The exact resource count can vary depending on the Terraform configuration and provider implementation.



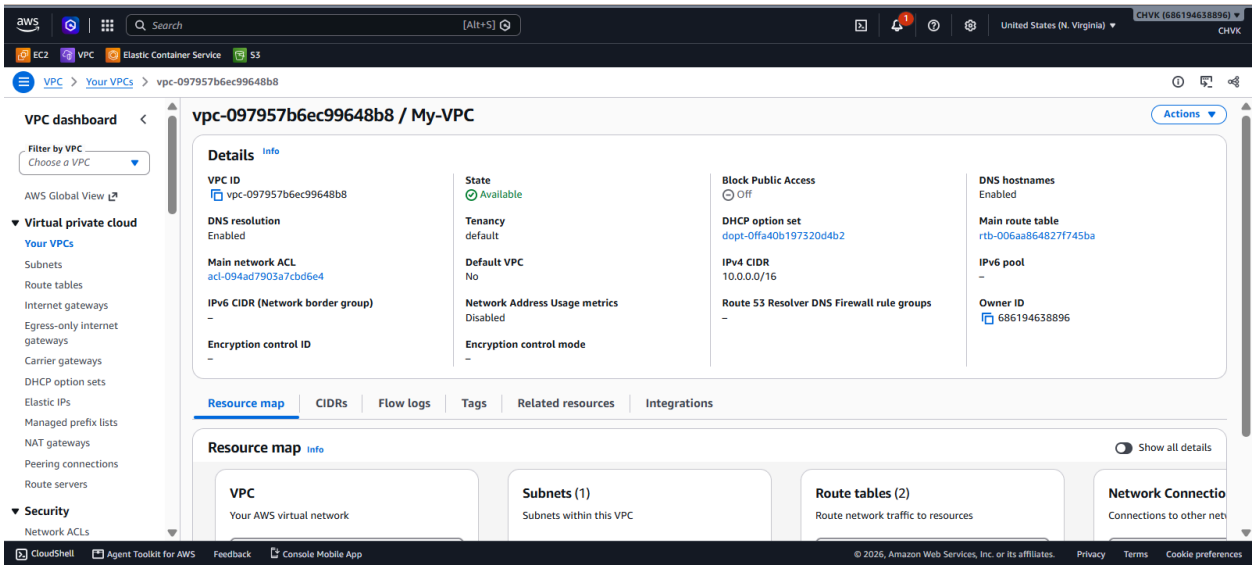
10. Verify Created AWS Resources

After successful `terraform apply`, we can verify the resources in AWS Console.

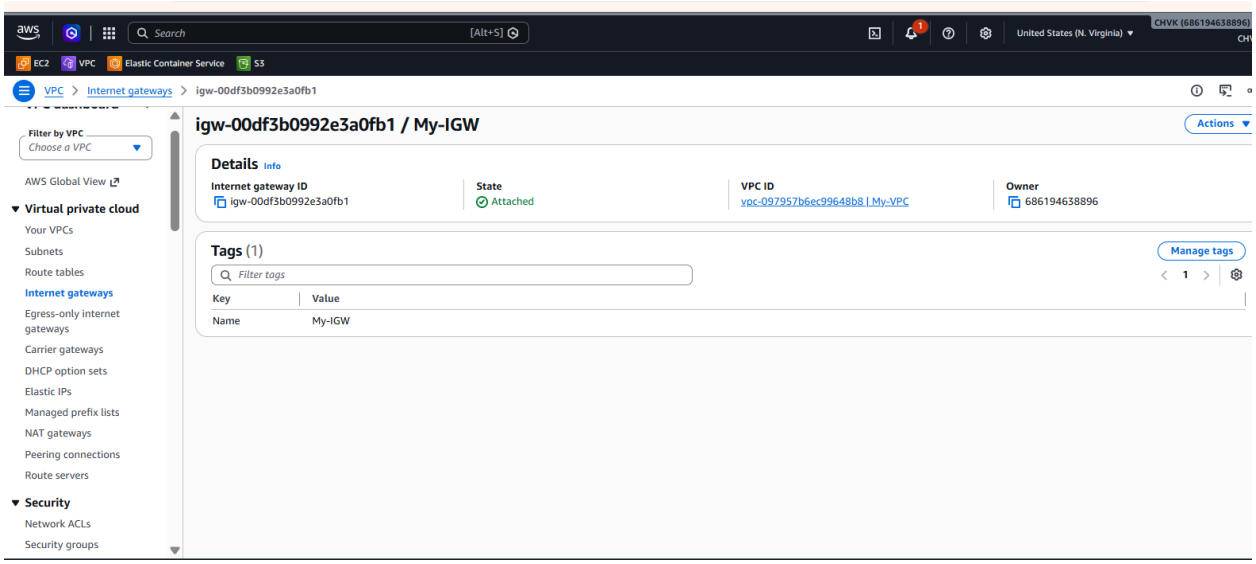
Go to AWS Console → VPC

Verify:

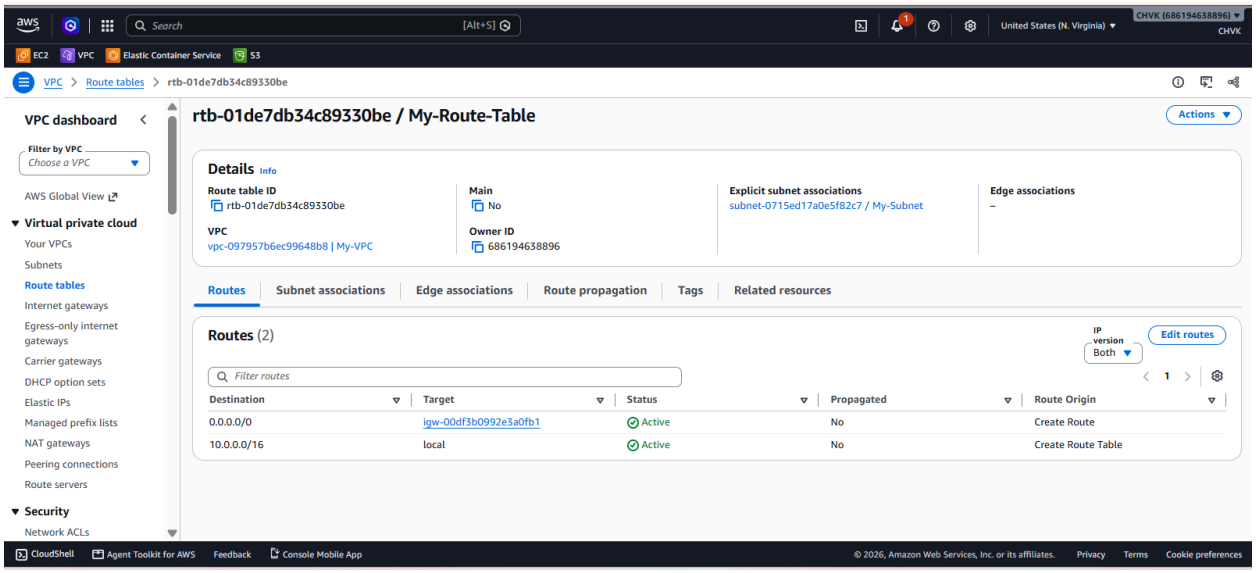
1. Your VPCs → `My-VPC`



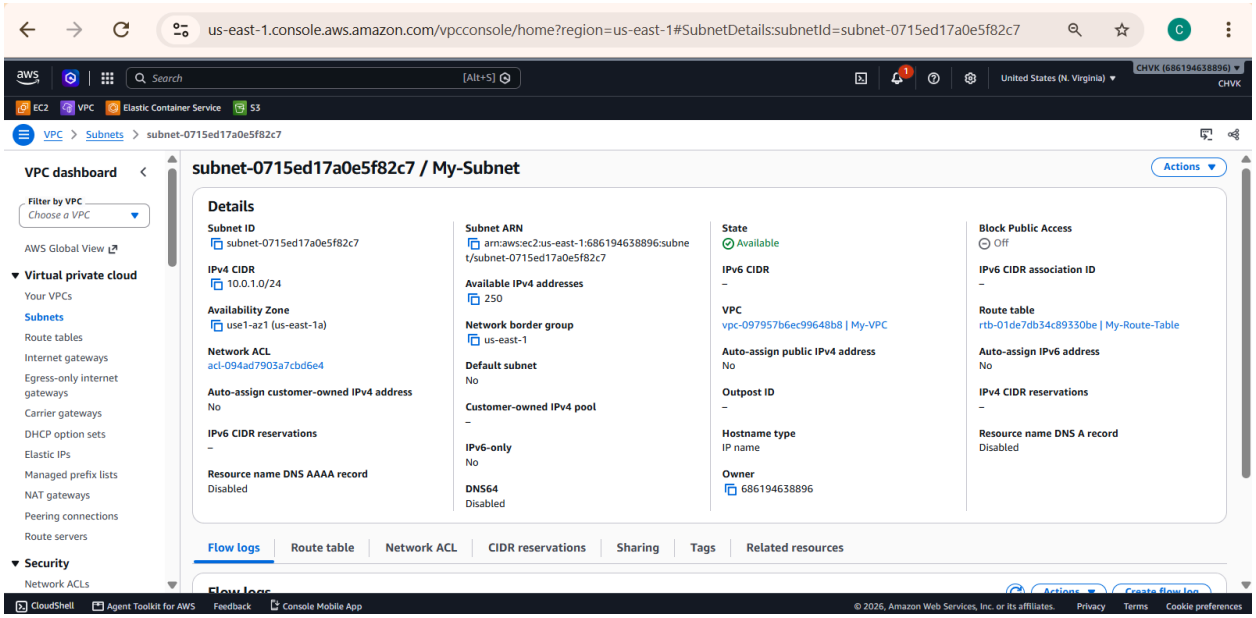
2. Internet Gateways → `My-IGW`



3. Route Tables → `My-Route-Table`



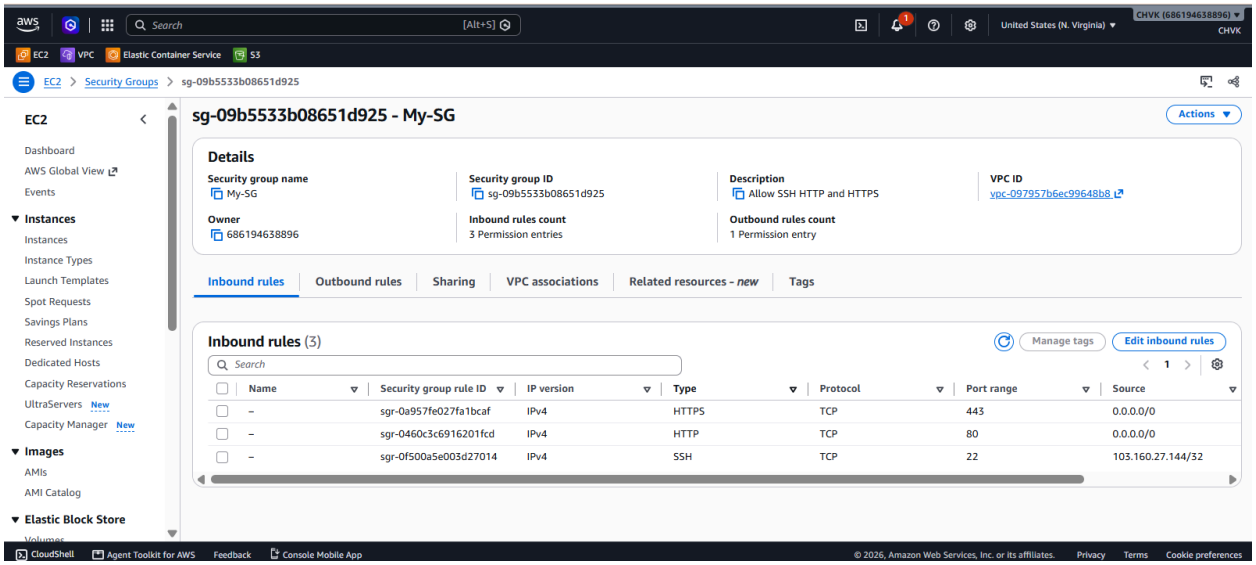
4. Subnets → `My-Subnet`



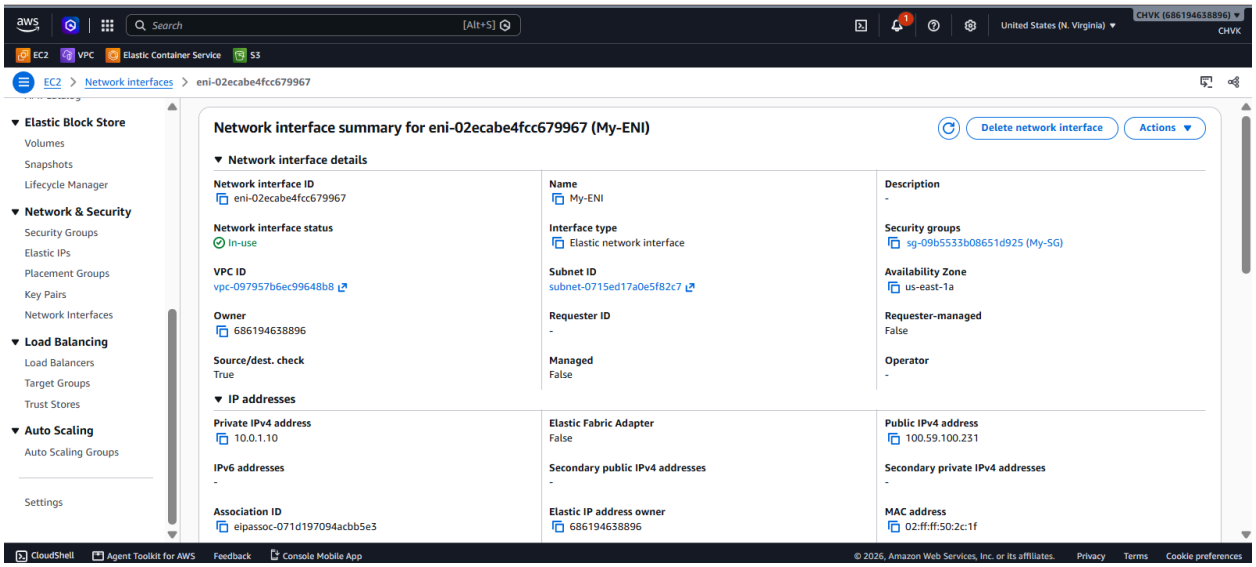
Go to AWS Console → EC2

Verify:

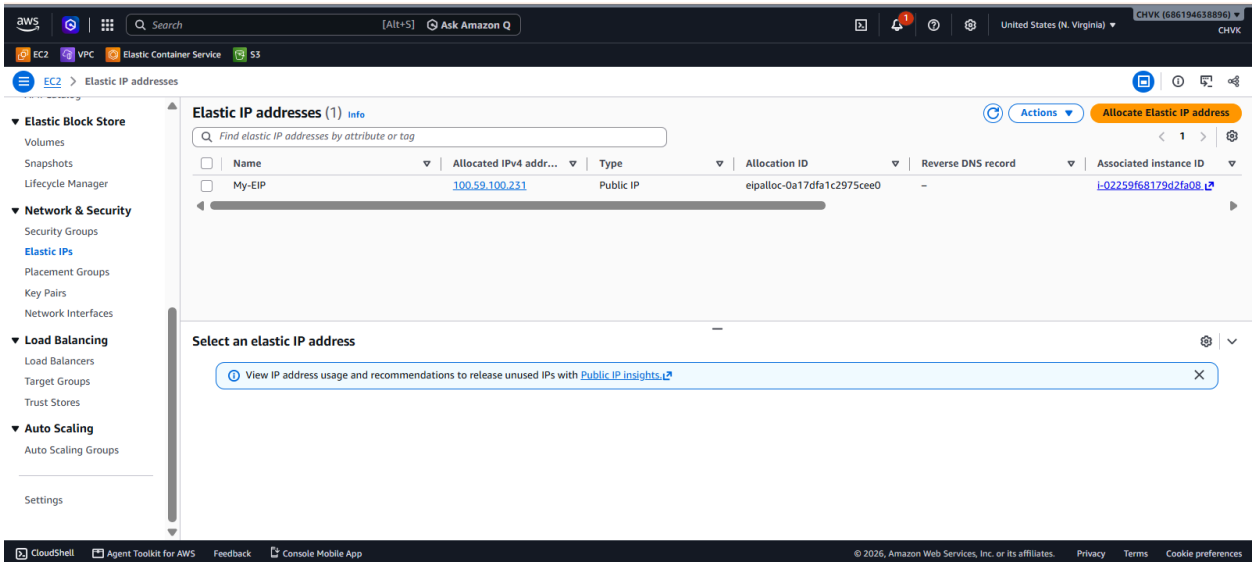
1. Security Groups → `My-SG`



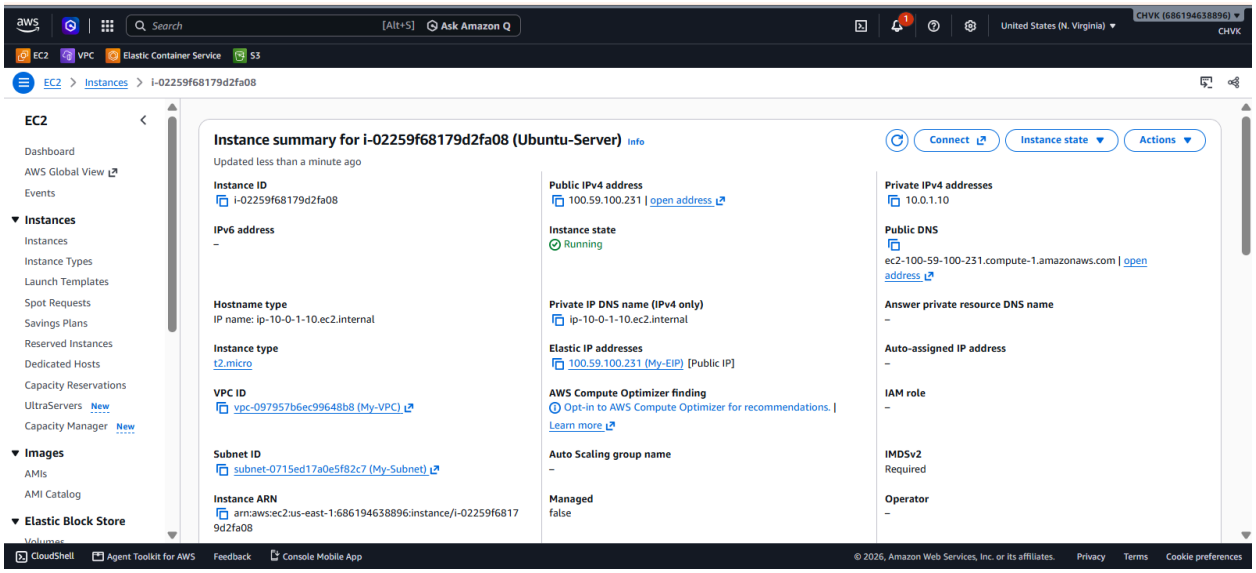
2. Network Interfaces → `My-ENI`



3. Elastic IPs → `My-EIP`



4. Instances → `Ubuntu-Server`



Expected Result

All resources should be created successfully, and the EC2 instance should be in the Running state.

11. Connect to Ubuntu EC2 Instance

We will connect to Ubuntu using SSH from Windows PowerShell.

Step 1: Open PowerShell.

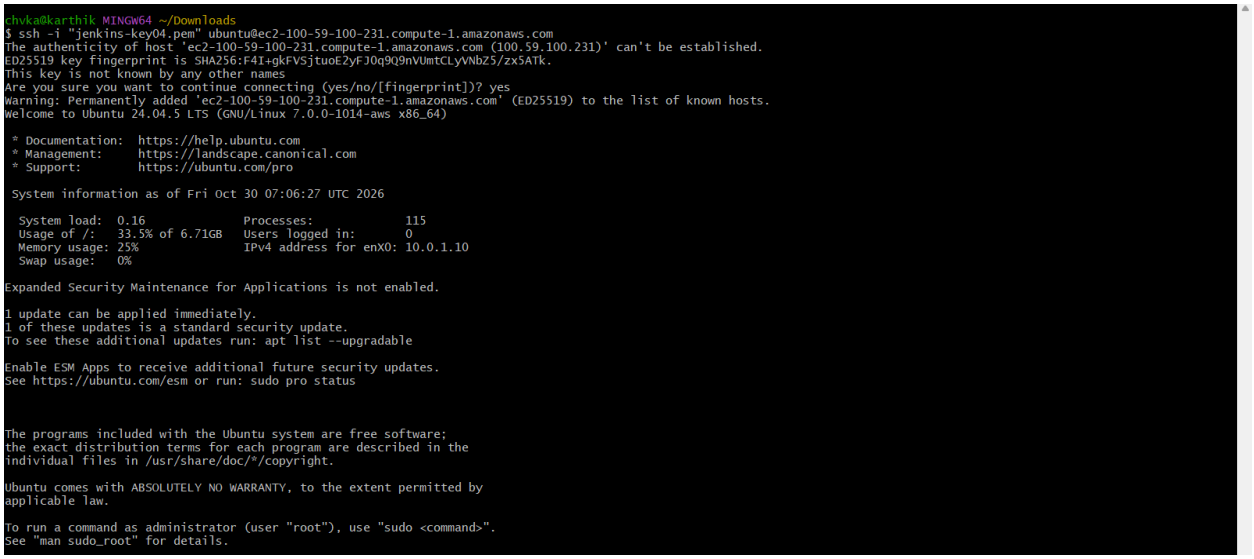
Step 2: Navigate to the folder containing your `.pem` key file.

```
None
cd Downloads
```

Step 3: Connect to the Ubuntu server.

```
None
ssh -i "your-key.pem" ubuntu@YOUR_ELASTIC_IP
```

Replace the key filename and Elastic IP with your actual values.



Step 4: Once connected, verify the operating system.

```
None
cat /etc/os-release
```

Expected Result: Ubuntu operating system information.

```
ubuntu@ip-10-0-1-10:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.5 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
ubuntu@ip-10-0-1-10:~$
```

12. Verify Apache2 Installation

Apache2 is installed automatically through Terraform's `user_data` script.

We do not need to install it manually.

Step 1: Check Apache2 status

```
None
sudo systemctl status apache2
```

Expected output:

```
None
Active: active (running)
```

```
ubuntu@ip-10-0-1-10:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-30 06:54:53 UTC; 12min ago
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 1895 (apache2)
      Tasks: 55 (limit: 657)
     Memory: 5.8M (peak: 6.0M)
        CPU: 70ms
     CGroup: /system.slice/apache2.service
             ├─1895 /usr/sbin/apache2 -k start
             ├─1897 /usr/sbin/apache2 -k start
             └─1898 /usr/sbin/apache2 -k start

Oct 30 06:54:53 ip-10-0-1-10 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Oct 30 06:54:53 ip-10-0-1-10 systemd[1]: Started apache2.service - The Apache HTTP Server.
ubuntu@ip-10-0-1-10:~$
```

Step 2: Check Apache2 service

```
None
sudo systemctl is-enabled apache2
```

Expected output:

```
None
enabled
```

```
ubuntu@ip-10-0-1-10:~$ sudo systemctl is-enabled apache2
enabled
ubuntu@ip-10-0-1-10:~$
```

Step 3: Test Apache locally

```
None
curl http://localhost
```

Expected result: Apache HTML page content.

```
ubuntu@ip-10-0-1-10:~$ curl http://localhost
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
  * {
    margin: 0px 0px 0px 0px;
    padding: 0px 0px 0px 0px;
  }

  body, html {
    padding: 3px 3px 3px 3px;

    background-color: #D8DBE2;

    font-family: Ubuntu, Verdana, sans-serif;
    font-size: 11pt;
    text-align: center;
  }

  div.main_page {
    position: relative;
    display: table;

    width: 800px;

    margin-bottom: 3px;
    margin-left: auto;
    margin-right: auto;
    padding: 0px 0px 0px 0px;

    border-width: 2px;
    border-color: #212738;
    border-style: solid;

    background-color: #FFFFFF;

    text-align: center;
  }
```

Step 4: Open Apache in browser

Copy the Elastic IP from Terraform output.

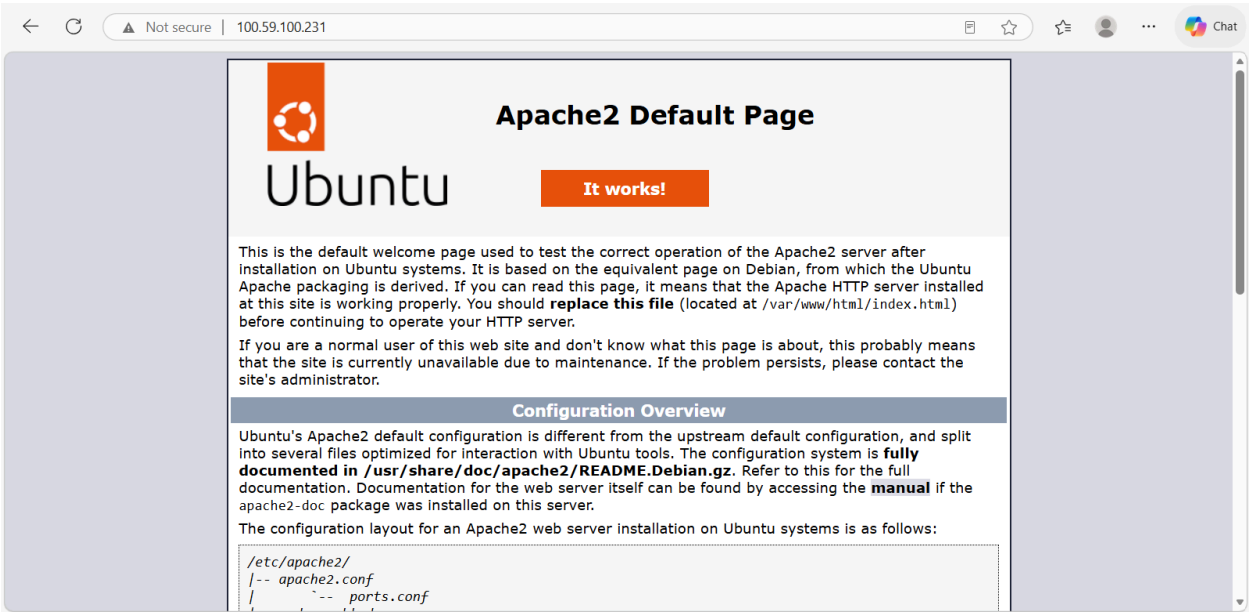
Open Chrome and enter:

```
None
http://YOUR_ELASTIC_IP
```

[Step-by-Step Guide: Configuring a Linux-Ubuntu Server on AWS Cloud | by Ryan Galazka | Medium
medium.com](#)

You should see the Apache2 Ubuntu Default Page.

Apache2 Successfully Running



13. Terraform State Verification

Run:

```
None
terraform state list
```

Purpose: Displays all resources managed by Terraform in the current state file.

You should see resources similar to:

```
None
aws_eip.my_eip
aws_instance.ubuntu_server
```

```
aws_internet_gateway.my_igw
aws_network_interface.my_eni
aws_route_table.my_route_table
aws_route_table_association.my_association
aws_security_group.my_sg
aws_subnet.my_subnet
aws_vpc.my_vpc
```

```
PS C:\terraform-workspaces\terraform-vpc-project> terraform state list
data.aws_ami.ubuntu
aws_eip.my_eip
aws_instance.ubuntu_server
aws_internet_gateway.my_igw
aws_network_interface.my_eni
aws_route_table.my_route_table
aws_route_table_association.my_association
aws_security_group.my_sg
aws_subnet.my_subnet
aws_vpc.my_vpc
PS C:\terraform-workspaces\terraform-vpc-project>
```

14. Destroy AWS Resources After Practice

When the practical is completed, execute:

```
None
terraform destroy
```

Terraform displays the resources that will be deleted.

Type:

```
None
yes
```

Purpose: Deletes the AWS resources created by this Terraform configuration, including the EC2 instance, Elastic IP, networking resources, and VPC.

This helps avoid unnecessary AWS charges. Keep your Terraform files if you want to recreate the infrastructure later.

```
Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_route_table_association.my_association: Destroying... [id=rtbassoc-0c805e030b0fbfca2]
aws_eip.my_eip: Destroying... [id=eipalloc-0a17dfa1c2975cee0]
aws_route_table_association.my_association: Destruction complete after 2s
aws_route_table.my_route_table: Destroying... [id=rtb-01de7db34c89330be]
aws_eip.my_eip: Destruction complete after 4s
aws_instance.ubuntu_server: Destroying... [id=i-02259f68179d2fa08]
aws_route_table.my_route_table: Destruction complete after 2s
aws_internet_gateway.my_igw: Destroying... [id=igw-00df3b0992e3a0fb1]
aws_internet_gateway.my_igw: Destruction complete after 2s
aws_instance.ubuntu_server: Still destroying... [id=i-02259f68179d2fa08, 00m10s elapsed]
aws_instance.ubuntu_server: Still destroying... [id=i-02259f68179d2fa08, 00m20s elapsed]
aws_instance.ubuntu_server: Destruction complete after 22s
aws_network_interface.my_eni: Destroying... [id=eni-02ecabe4fcc679967]
aws_network_interface.my_eni: Destruction complete after 1s
aws_subnet.my_subnet: Destroying... [id=subnet-0715ed17a0e5f82c7]
aws_security_group.my_sg: Destroying... [id=sg-09b5533b08651d925]
aws_subnet.my_subnet: Destruction complete after 1s
aws_security_group.my_sg: Destruction complete after 1s
aws_vpc.my_vpc: Destroying... [id=vpc-097957b6ec99648b8]
aws_vpc.my_vpc: Destruction complete after 1s

Destroy complete! Resources: 9 destroyed.
```

15. Final Task Completion Summary

No.	Assignment Task	Terraform Resource
1	Create VPC	`aws_vpc`
2	Create Internet Gateway	`aws_internet_gateway`
3	Create Custom Route Table	`aws_route_table`
4	Create Subnet	`aws_subnet`

5	Associate Subnet with Route Table	`aws_route_table_association`
6	Allow ports 22, 80, 443	`aws_security_group`
7	Create Network Interface	`aws_network_interface`
8	Assign Elastic IP	`aws_eip`
9	Create Ubuntu and install Apache2	`aws_instance` + `user_data`

16. Conclusion

Successfully provisioned AWS networking infrastructure and an Ubuntu EC2 instance using Terraform. Created a custom VPC, Internet Gateway, Route Table, Subnet, Security Group, Network Interface, and Elastic IP. Configured the Ubuntu server to automatically install and enable Apache2 using Terraform user data. Finally, verified the Apache2 web server through a browser using the Elastic IP address.

Important: This configuration creates a public-facing HTTP server. Port 443 is allowed, but HTTPS is not configured yet. Also, keep your AWS credentials and `.pem` private key secure, and do not upload `terraform.tfvars` or Terraform state files containing sensitive information to GitHub.